Index Construction

Academic Year 2025/2026

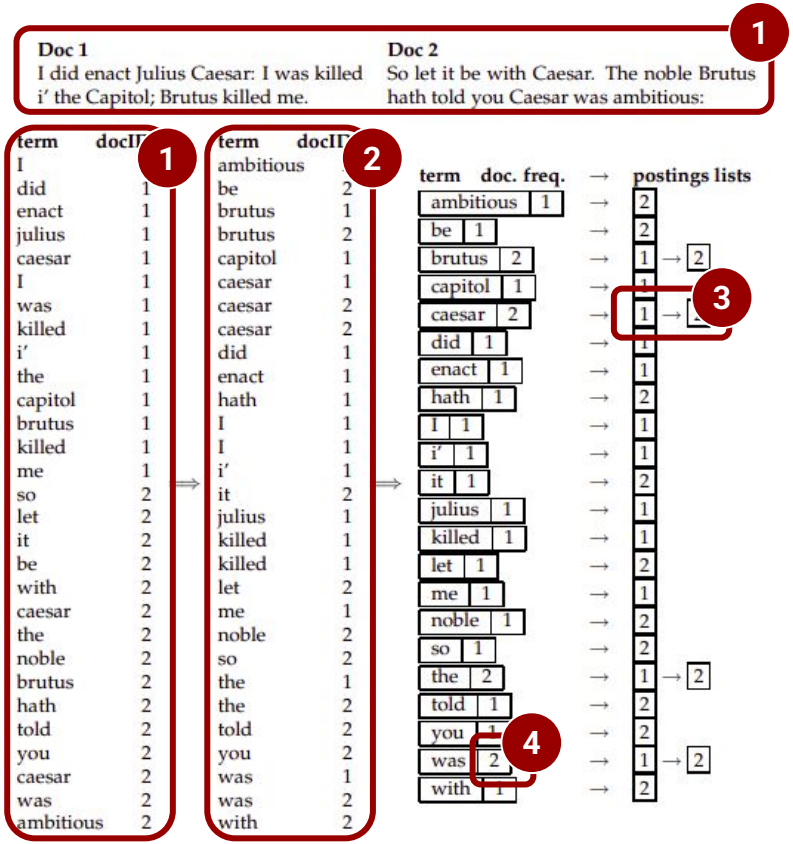
Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
Master Degree in Computer Engineering

Building an Index

How already discussed, building an index requires:

1. **Scanning** all documents and assembling all term–docID pairs
2. **Sorting** the pairs with the term as primary key and the DocID as secondary
3. **Organizing** all DocIDs for each term into a Postings List
4. **Computing** statistics like term and document frequency



In-memory vs Disk-saved implementation

A Standard Inverted Index can be built and **kept in memory** only if the number of documents and terms is quite limited.

Nowadays it is easy to have a collection of documents which cannot be kept in memory.

An in-memory indexer **does not scale at all**.

We need to keep data on the disk, but writing and reading data from disk is **very very slow**

Blocked Sort-Based Indexing Algorithm

The solution is to use the **BSBI Algorithm**.

This algorithm aims to reduce the **number of disk seeks** during sorting.

To make the algorithm **more efficient**, instead of using Terms, it assigns each Term to a **TermId**. A TermID and a DocID are represented by an integer (4 bytes), so, for example, having a collection of documents with 100 million of tokens (remind: a token is a sequence of characters semantically grouped together) we need **0.8GB of RAM** to keep all pairs (TermID, DocID) in memory.

Blocked Sort-Based Indexing Algorithm

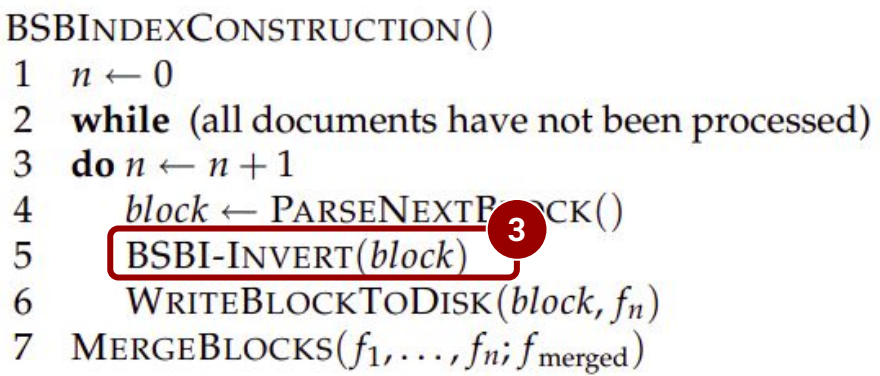
1. We define a size for a block. We choose the block size to fit comfortably into memory to permit a **fast in-memory sort**
2. Line 4 retrieves the next block to be parsed. ***Block*** contains all termID-DocID pairs which fit into the memory

```
BSBINDEXCONSTRUCTION()
1  n ← 0
2  while (all documents have not been processed)
3  do n ← n + 1
4     block ← PARSENEXTBLOCK()
5     BSBI-INVERT(block)
6     WRITEBLOCKTODISK(block, f_n)
7  MERGEBLOCKS(f_1, ..., f_n; f_merged)
```

2

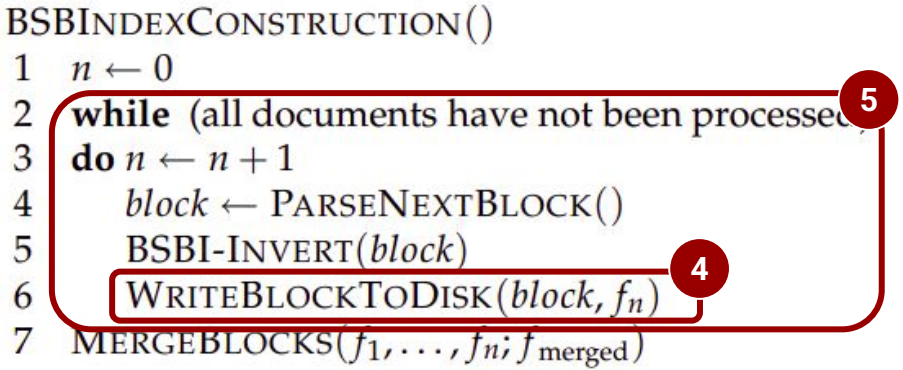
Blocked Sort-Based Indexing Algorithm

3. Line 5 builds the inverted index for the terms in the block. This means:
   a. **Sorting** all termID-DocID (first on the TermID, then on the DocID)
   b. **Organizing** all DocIDs for each term into a Postings List
   c. **Computing** statistics like term and document frequency



Blocked Sort-Based Indexing Algorithm

4. Line 6 writes intermediate sorted results on disk on a different file (related to the processed block). *fn* is a **partial inverted index**.
5. Repeating 4-6 till all documents are not processed



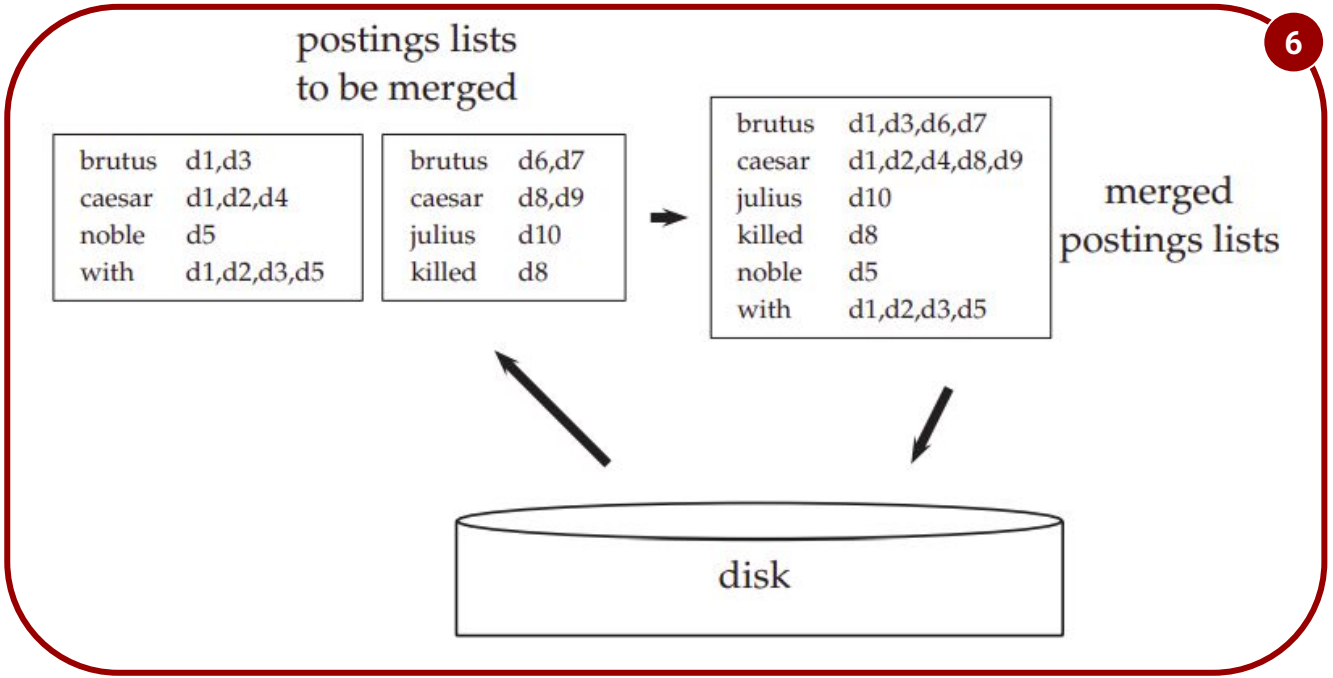
Blocked Sort-Based Indexing Algorithm

6. Line 7 reads all *n* written files (representing the partial inverted index) and merge results all together into the final inverted index (saved back to the disk)

```
BSBINDEXCONSTRUCTION()
1  n ← 0
2  while (all documents have not been processed)
3  do n ← n + 1
4     block ← PARSENEXTBLOCK()
5     BSBI-INVERT(block)
6     WRITEBLOCKTODISK(block, f_n)
7  MERGEBLOCKS(f_1, ..., f_n; f_merged)
```

6

Blocked Sort-Based Indexing Algorithm



Blocked Sort-Based Indexing Algorithm

Let’s have a deep dive into the **Merging Operation**. How we can do this operation in an efficient way?

1. We **open all *n* files** containing a partial inverted index (remind that all terms are already sorted)
2. We **use a pointer** to the next Term not processed yet (*n* pointers for *n* files)
3. In each iteration, we select the lowest Term that has not been processed yet
4. All postings lists for this term are read and merged
5. The merged list is written back to disk

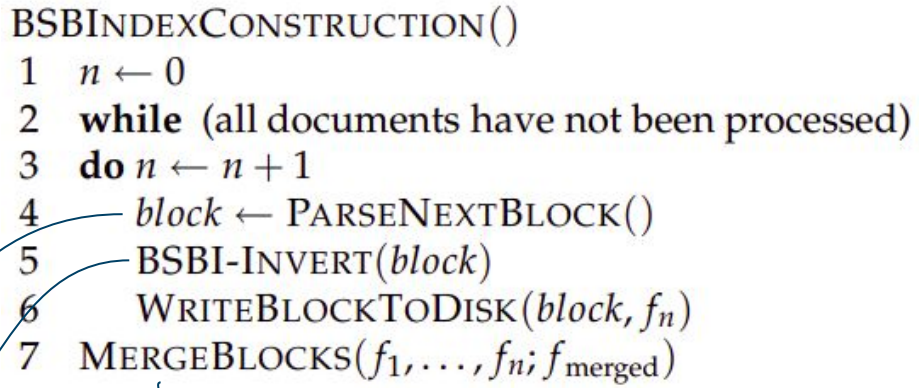
Complexity of BSBI Algorithm

Let *T* be the **number of tokens**

The complexity time for the algorithm is **O(T logT)**

The algorithm spends more time in

- parsing the documents
- sorting (TermID, DocID) pairs
- merging all partial blocks



Drawbacks of the BSBI Algorithm

- Each pair makes use of TermID. This means that we need a data structure for mapping Terms into TermIDs. For very large collections, this data structure **does not fit into memory**.
- It requires **sorting** all (TermID, DocID) pairs

Single-pass in-memory indexing

The SPIMI algorithm is more efficient than the Blocked Sort-Based Indexing Algorithm because:

- It uses Terms instead of TermID, so it does not need any data structure for mappings
- It does not require sorting (Term, DocID) pairs (which is done by BSBI-Invert function in BSBI Algorithm)
    - Sorting operation is postponed and applied on the dictionary

Single-pass in-memory indexing

```
SPIMI-INVERT(token_stream)
 1  output_file = NEWFILE()
 2  dictionary = NEWHASH()
 3  while (free memory available)
 4  do token ← next(token_stream)
 5     if term(token) ∉ dictionary
 6        then postings_list = ADDTODICTIONARY(dictionary, term(token))
 7        else postings_list = GETPOSTINGSLIST(dictionary, term(token))
 8     if full(postings_list)
 9        then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms ← SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
```

- *token_stream*: a collection of (Token, DocID) pairs
- *dictionary* is an Hash Table

Single-pass in-memory indexing

```
SPIMI-INVERT(token_stream)
 1  output_file = NEWFILE()
 2  dictionary = NEWHASH()
 3  while (free memory available)
 4  do token ← next(token_stream)
 5     if term(token) ∉ dictionary
 6       then postings_list = ADDTODICTIONARY(dictionary, term(token))
 7       else postings_list = GETPOSTINGSLIST(dictionary, term(token))
 8     if full(postings_list)
 9       then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms ← SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
```

- Tokens are processed one by one
- When a term occurs for the first time, it is added to the dictionary. The corresponding (empty) Postings List is returned

Single-pass in-memory indexing

```
SPIMI-INVERT(token_stream)
 1  output_file = NEWFILE()
 2  dictionary = NEWHASH()
 3  while (free memory available)
 4  do token ← next(token_stream)
 5     if term(token) ∉ dictionary
 6       then postings_list = ADDTODICTIONARY(dictionary, term(token))
 7       else postings_list = GETPOSTINGSLIST(dictionary, term(token))
 8     if full(postings_list)
 9       then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms ← SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
```

- If the term is already in the dictionary, the corresponding Postings List is returned
- Using Hash Table, managing Dictionary is very efficient

Single-pass in-memory indexing

```
SPIMI-INVERT(token_stream)
 1  output_file = NEWFILE()
 2  dictionary = NEWHASH()
 3  while (free memory available)
 4  do token ← next(token_stream)
 5     if term(token) ∉ dictionary
 6        then postings_list = ADDTODICTIONARY(dictionary, term(token))
 7        else postings_list = GETPOSTINGSLIST(dictionary, term(token))
 8     if full(postings_list)
 9        then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms ← SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
```

- The Posting List is managed like a List having predefined size. This instruction verifies if the List is full (all available seats for terms are busy). If so, the size of the is doubled.

Single-pass in-memory indexing

```
SPIMI-INVERT(token_stream)
 1  output_file = NEWFILE()
 2  dictionary = NEWHASH()
 3  while (free memory available)
 4  do token ← next(token_stream)
 5     if term(token) ∉ dictionary
 6        then postings_list = ADDTODICTIONARY(dictionary, term(token))
 7        else postings_list = GETPOSTINGSLIST(dictionary, term(token))
 8     if full(postings_list)
 9        then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms ← SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
```

The Document ID is added to the Postings List.

Single-pass in-memory indexing

```
SPIMI-INVERT(token_stream)
 1  output_file = NEWFILE()
 2  dictionary = NEWHASH()
 3  while (free memory available)
 4  do token ← next(token_stream)
 5     if term(token) ∉ dictionary
 6        then postings_list = ADDTODICTIONARY(dictionary, term(token))
 7        else postings_list = GETPOSTINGSLIST(dictionary, term(token))
 8     if full(postings_list)
 9        then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms ← SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
```

Processing Terms continues until we still have memory available (Posting List and Dictionary grow and they can fill all memory)

Single-pass in-memory indexing

```
SPIMI-INVERT(token_stream)
 1  output_file = NEWFILE()
 2  dictionary = NEWHASH()
 3  while (free memory available)
 4  do token ← next(token_stream)
 5     if term(token) ∉ dictionary
 6        then postings_list = ADDTODICTIONARY(dictionary, term(token))
 7        else postings_list = GETPOSTINGSLIST(dictionary, term(token))
 8     if full(postings_list)
 9        then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms ← SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
```

When memory is exhausted 1) we sort the terms in the dictionary (for the final merging operation of all blocks) and 2) write the block on the disk

Distributed Indexing

When the number of documents to process is very high, search engines use **distributed indexing algorithms** for index construction.

The result of the construction process is a distributed index that is partitioned **across several machines**.

Distributed Indexing

Building a distributed index requires a **cluster of machines** cooperating all together.

The job is **splitted into chunks** and **distributed** among all **workers**

A **master** directs the process of assigning **task**

Distributed Indexing

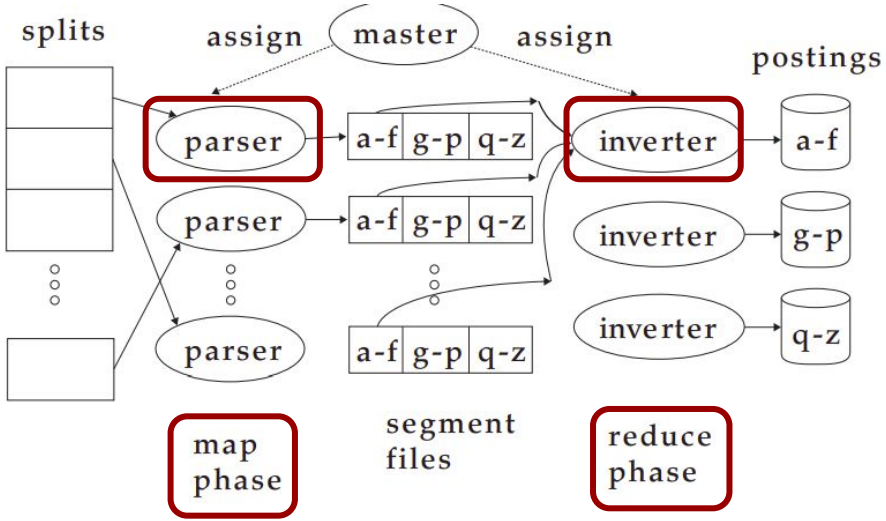
- The master process splits collection of documents into *n* **chunks**
- Each chunk should not be **neither too small nor too large** (16MB or 64MB are good sizes in distributed indexing)
- Each chunk is sent to a worker. Chunks are **not preassigned**
- As soon as a process finishes processing one chunk, the master assigns the next one to it
- If a machine **dies due to hardware problems**, the chunk it was working on is simply reassigned to another machine.
- The strategy adopted by the workers is based on the **MapReduce Algorithm**

Distributed Indexing

MapReduce: it is a programming strategy for computing huge amount of data in parallel

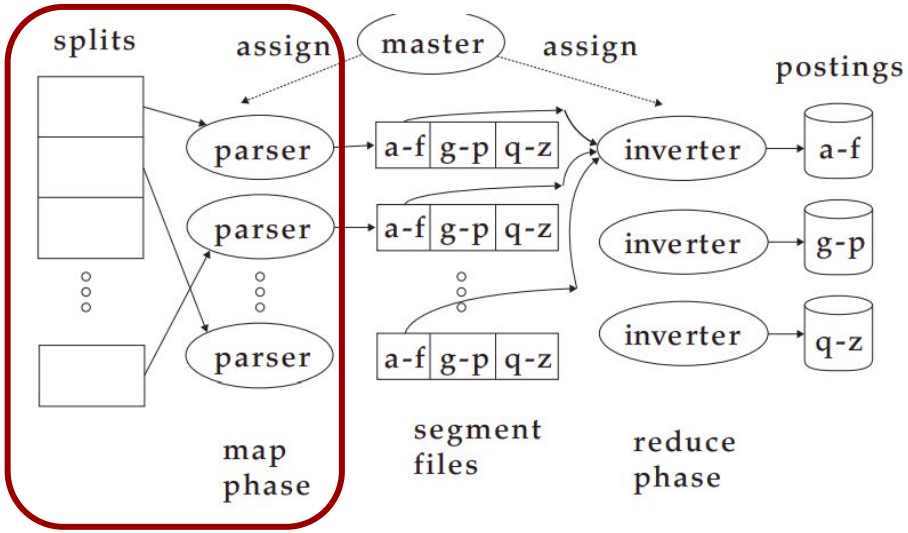
Programmers never write a MapReduce application. They just write two functions: **map and reduce functions**

Workers executing map/reduce phase are usually called **parser/inverter**



Distributed Indexing

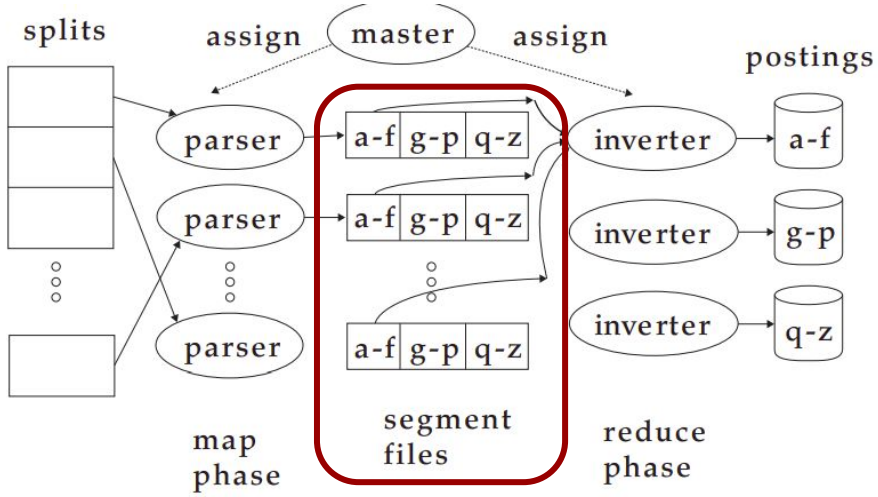
Map phase: operates on the input data (the chunk of web pages) and generates **key-value pairs** having the form of (TermID-DocID). Of course all workers must share a consistent mapping of Term and TermID



Distributed Indexing

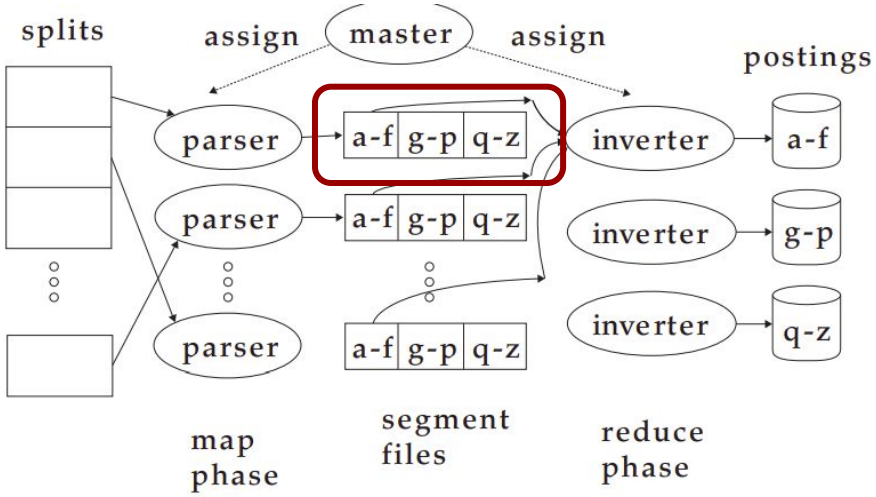
Map phase: the parser starts processing the chunk received by the master. It creates pairs of **key-value**, and then stores them into local intermediate file called **segment files.**

Each segment file is a partition of key-value pairs



Distributed Indexing

Map phase: we want all values for a given key to be stored close together, such that for a given key we can have the list of all documents containing that key. To do so in parallel, all the keys **are partitioned** in *j* parts (in the pictures all keys have been partitioned in 3 parts according to the first letter (a-f), (g-p) and (q-z), but there might be several different ways to partition all keys

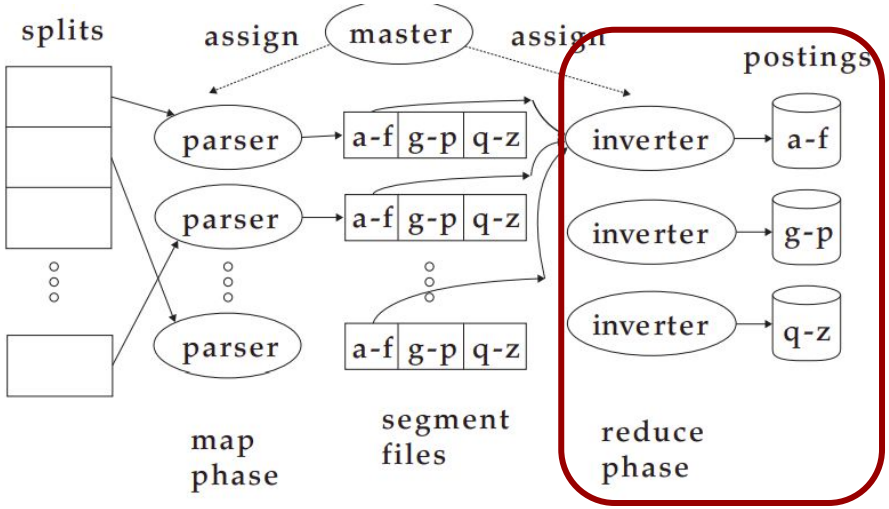


Distributed Indexing

Reduce phase: collecting all values for a given key into one list is the task of the **inverter.** The master assigns each term partition to a different inverter.

In case of **failure**, the task can be reassigned to another inverter.

Finally, the list of values is sorted for each key and written to the final sorted **postings list**



Term-partitioned Distributed Indexing

Parsers and Inverters are not separate sets of machines.

The master identifies **idle machines** and assigns tasks to them.

The same machine **can be both parser and inverter** during map phase and reduce phase.

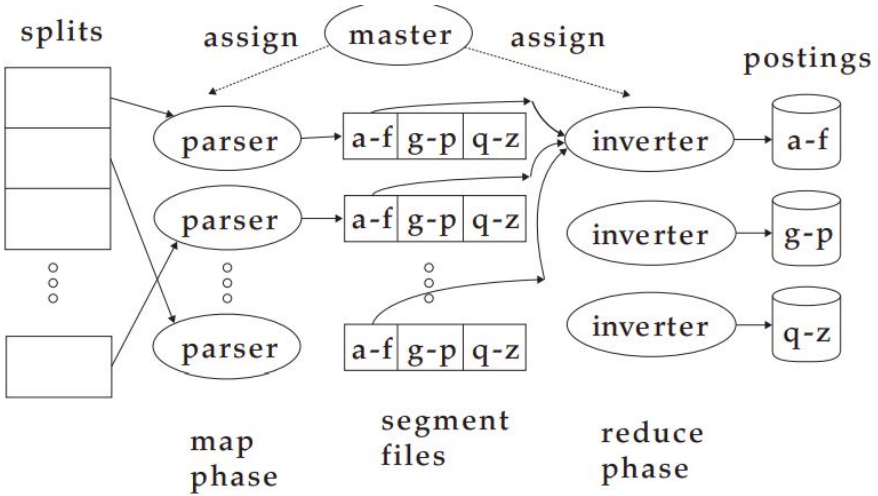
The distributed index described so far is named **term-partitioned distributed index**

Problems using distributed indexes

For example if I'm looking for "apple" I'll send the search only to the node containing the posting list [A-F]

But there are two big problems that we need to face: **Multi-term queries**

A possible solution would be to **share postings lists** between sets of nodes for merging, but this can be very **expensive in terms of Network Traffic and Computation**



Document-partitioned Distributed Indexing

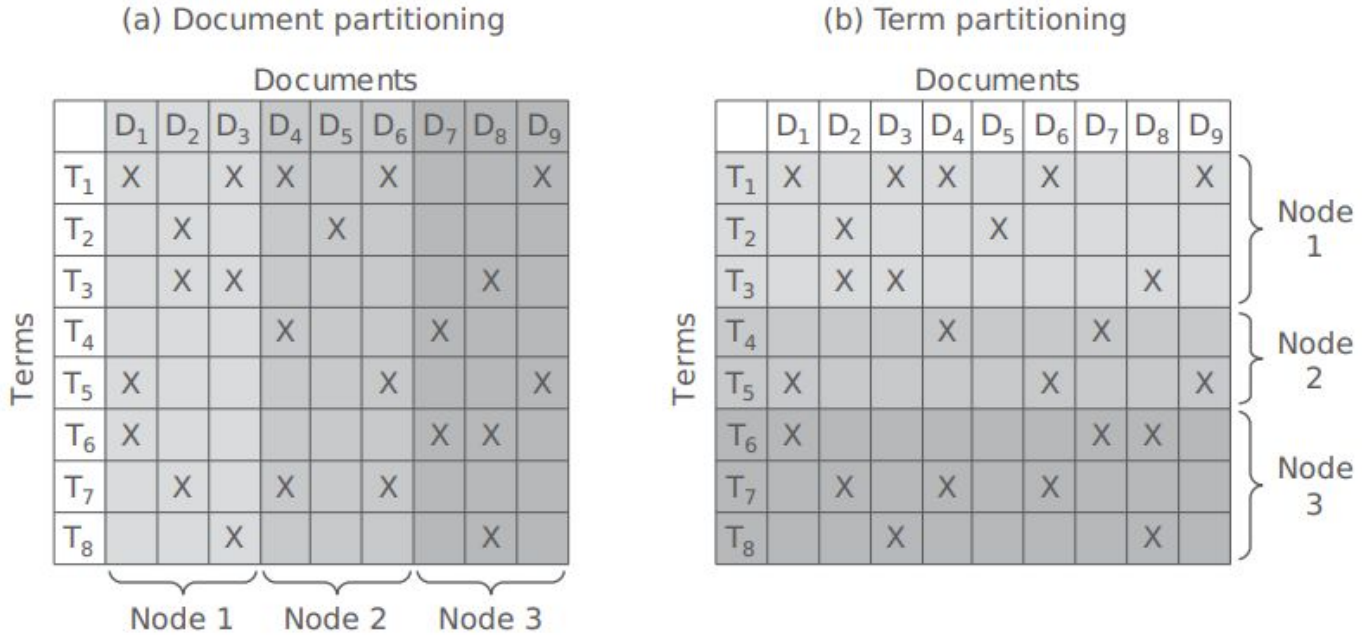
Why not to use a document-partitioned distributed index?

- Each node contains the index for a subset of all documents.
- Each query is distributed to all nodes.
- The results from various nodes are merged before presentation to the user.

This approach makes **global statistics** used in scoring harder to be computed, because they should be computed across the entire document collection (but each node contains only a small subset of all documents).

For such a kind of global statistics, **a distributed background process** is usually used. The process refreshes periodically the node indexes with fresh global statistics

Document Partitioning vs Term Partitioning



Dynamic Indexing

Document collection can **change over the time**. New documents can be added, existing documents can be updated or modified.

An easy way to manage changes in the Collection is to build a **new index** while the old one is available only for queries until the new is ready.

Of course, building a new index can be **time consuming** and requires computational resources.

Dynamic Indexing

If we need that the document is included quickly in the index, we can maintain **two indexes**:

1. the **main index** (quite large)
2. an **auxiliary index** that stores new or updated documents. This, being **small**, is kept in memory

Searches are run on both indexes and **results are merged**.

Document deletions are managed by using an **invalidation bit vector**. Results related to **deleted documents are filtered**

Dynamic Indexing

When the auxiliary index is too large, it **is merged** with the main index and then stored into the disk.

Merging the auxiliary index with the main index means **merging posting lists**.

Merging can be time consuming, depending on the number of files and their size used to store the main index.

Logarithmic Merging

An efficient algorithm for merging indexes is the **Logarithmic Merging**.

Let *T* be the number of total postings

Let *n* be the maximum number of postings to be stored into the auxiliary index.

The idea behind the Logarithmic Merging Algorithm is to create $k = \log_2(T/n)$ indexes, named $I_0, I_1, I_2, \ldots, I_{k-1}$ having size (number of postings to be stored) respectively $2^0n, 2^1n, 2^2n, \ldots, 2^{k-1}n$ which are **permanently stored** into disk, and an **auxiliary index** Z_0 having size (number of posting to be stored) equals to *n*. The auxiliary index is kept **in memory**.

Logarithmic Merging

Note that

$$2^0 n + 2^1 n + 2^2 n + \ldots + 2^{k-1} n = \sum_{i=0}^{k-1} 2^i n$$

$$= \frac{n(2^k - 1)}{2 - 1} = n(2^k - 1)$$

$$= n(2^{log_2 \frac{T}{n}} - 1) = n(\frac{T}{n} - 1) = T - n$$

This means that all persistent indexes are able to store only *T* - *n* postings, because the remainder *n* are kept into the in-memory index

Logarithmic Merging

The Logarithmic Merging works as follows:

- at the beginning, the first *n* postings are accumulated into Z_0 (in-memory auxiliary index)
- When the limit *n* is reached, all postings in Z_0 are transferred to a new persistent I_0. Z_0 is then set empty.
- Z_0 in then filled again and when the limit is reached again, Z_0 should be transferred to I0 but this persistent index is not empty so Z_0 and I_0 are merged together. The result is temporary stored into Z_1 and then moved in I_1. The index I_0 is then set empty.

Logarithmic Merging

First Few Steps

n	$2^0 \times n$	$2^1 \times n$	$2^2 \times n$	$2^3 \times n$	...
Z_0 $Z_0 \to I_0$	I_0				
Z_0	I_0 $Z_0 \cup I_0 \to Z_1 \to I_1$	I_1			
Z_0 $Z_0 \to I_0$	I_0	I_1 I_1 I_1			
Z_0	I_0 $Z_0 \cup I_0 \to Z_1$	I_1 I_1 $Z_1 \cup I_1 \to Z_2 \to I_2$	I_2		

Logarithmic Merging

- Auxiliary Index Z_0 is empty
- The set of all persistent indexes is empty

```
LOGARITHMICMERGE()
1  Z_0 ← ∅    (Z_0 is the in-memory index.)
2  indexes ← ∅
3  while true
4  do LMERGEADDTOKEN(indexes, Z_0, GETNEXTTOKEN())
```

Logarithmic Merging

One token at a time is added to the auxiliary index

```
LOGARITHMICMERGE()
1  Z_0 ← ∅   (Z_0 is the in-memory index.)
2  indexes ← ∅
3  while true
4  do LMERGEADDTOKEN(indexes, Z_0, GETNEXTTOKEN())
```

Logarithmic Merging

```
LMERGEADDTOKEN(indexes, Z_0, token)
 1  Z_0 ← MERGE(Z_0, {token})
 2  if |Z_0| = n
 3     then for i ← 0 to ∞
 4          do if I_i ∈ indexes
 5                then Z_{i+1} ← MERGE(I_i, Z_i)
 6                     (Z_{i+1} is a temporary index on disk.)
 7                     indexes ← indexes − {I_i}
 8                else I_i ← Z_i    (Z_i becomes the permanent index I_i.)
 9                     indexes ← indexes ∪ {I_i}
10                     BREAK
11          Z_0 ← ∅
```

The new token is added to the auxiliary index

Logarithmic Merging

Temporary and persistent indexes are created

```
LMERGEADDTOKEN(indexes, Z_0, token)
 1  Z_0 ← MERGE(Z_0, {token})
 2  if |Z_0| = n
 3    then for i ← 0 to ∞
 4         do if I_i ∈ indexes
 5              then Z_{i+1} ← MERGE(I_i, Z_i)
 6                   (Z_{i+1} is a temporary index on disk.)
 7                   indexes ← indexes − {I_i}
 8              else I_i ← Z_i    (Z_i becomes the permanent index I_i.)
 9                   indexes ← indexes ∪ {I_i}
10                   BREAK
11         Z_0 ← ∅
```

Logarithmic Merging

- Index Construction Time = O(T log(T/n))
- Logarithmic Merging is more efficient in merging but
  - **query processing** now requires the merging of O(log T/n) indexes, whereas it is O(1) if we just have a main and auxiliary index